

Machine Learning

8. *Unsupervised Learning*

Sunwoo Lee
sunwool@inha.ac.kr



INHA UNIVERSITY

Today, we will learn...

- Unsupervised Learning
 - What is 'unsupervised' learning?
 - Clustering Algorithm: K-Means & DBSCAN



Do not share or distribute these slides!

저작권 안내 / Copyright Notice

본 사이트에서 수업 자료로 이용되는 저작물은 저작권법 제25조 수업목적저작물 이용 보상금제도의 의거, 한국복제전송저작권협회와 약정을 체결하고 적법하게 이용하고 있습니다. **약정범위를 초과하는 사용은 저작권법에 저촉될 수 있으므로 수업자료의 대중 공개·공유 및 수업 목적외의 사용을 금지합니다. (일부 공개로 제공되는 유튜브 강의를 불법으로 공유하거나 배포하는 행위를 금함)**

The teaching materials provided on this site are copyrighted and signed legally by the Korea Reproduction and Transmission Rights Association. Using these materials beyond the scope of the agreement is a violation against copyright laws. **Hence, one is not allowed to open or share the materials in public. Using them outside of the class is strictly prohibited.**

2020. 3. 11.

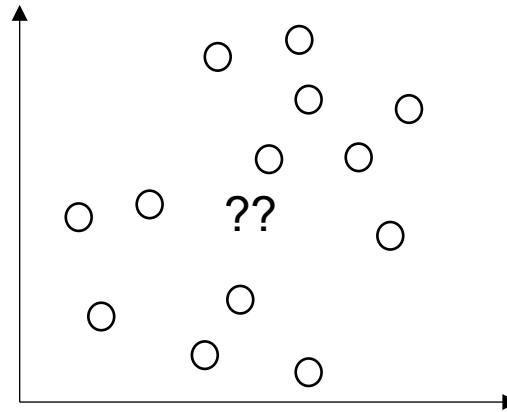
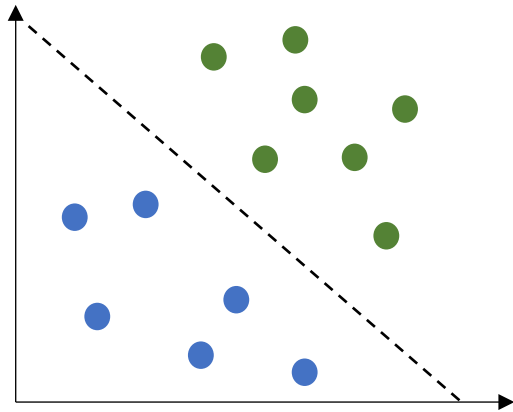
인하대학교·한국복제전송저작권협회

Inha University · Korea Reproduction and Transmission Rights Association

- ***Unsupervised Learning?***
- Clustering
- Example: K-Means
- Example: DBSCAN
- Example: Hierarchical Clustering

Unsupervised Learning

- A program learns inherent data patterns *without the need for human intervention* (no labels).
- Usually, unsupervised learning algorithms *automatically* find data structures, patterns, or any types of hidden knowledge out of the provided data.

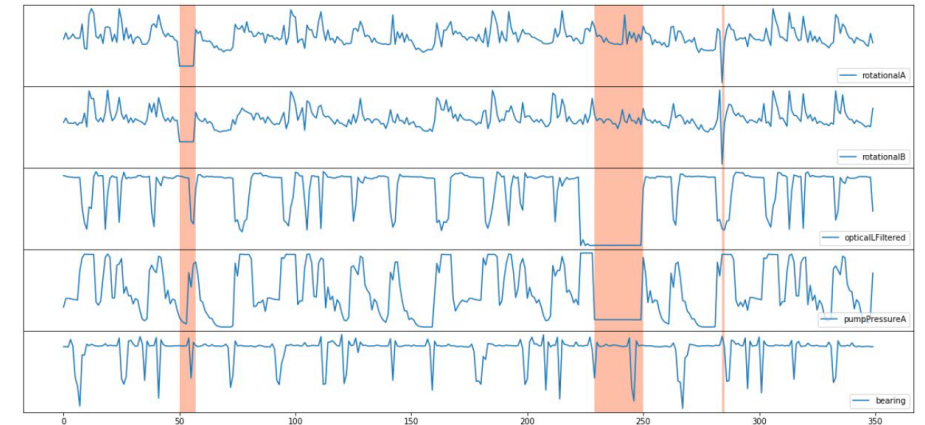


Real-World Applications

Image Segmentation

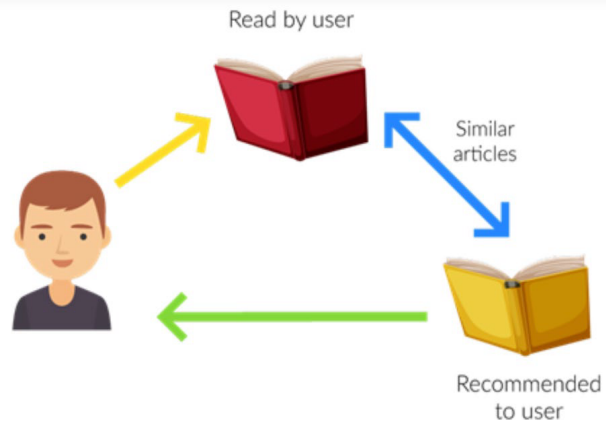


Anomaly Detection



Auto Engine Inspection Data

Recommender Systems



완벽한 정답을 기대할 수도 없고 기대하지도 않는 상황에 사용 가능

Why Is It Important?

- In this Big Data era, data is constantly getting generated everywhere (mobile phones, laptops, IoT, etc).
- If you ignore unlabeled data, you are missing 99.999999% of the available data! 자연계 대다수 데이터는 레이블이 없음, 레이블을 다는 작업은 상당한 시간과 비용 필요
- The key challenge is how to exploit the available raw data!



Imagine how many people are making new data every single second!

Unsupervised Learning Algorithms

- ***Clustering***
 - K-Means, DBSCAN, Hierarchical clustering, etc.
- Dimension Reduction
 - Principal Component Analysis (PCA), Autoencoders, etc.

We will focus on this 'clustering' algorithm in this lecture.

- Unsupervised Learning?
- ***Clustering***
- Example: K-Means
- Example: DBSCAN
- Example: Hierarchical Clustering

Clustering

- Grouping the given data samples to a few distinct subsets.



I only want
Mandoo...

Clustering Metric: Distance

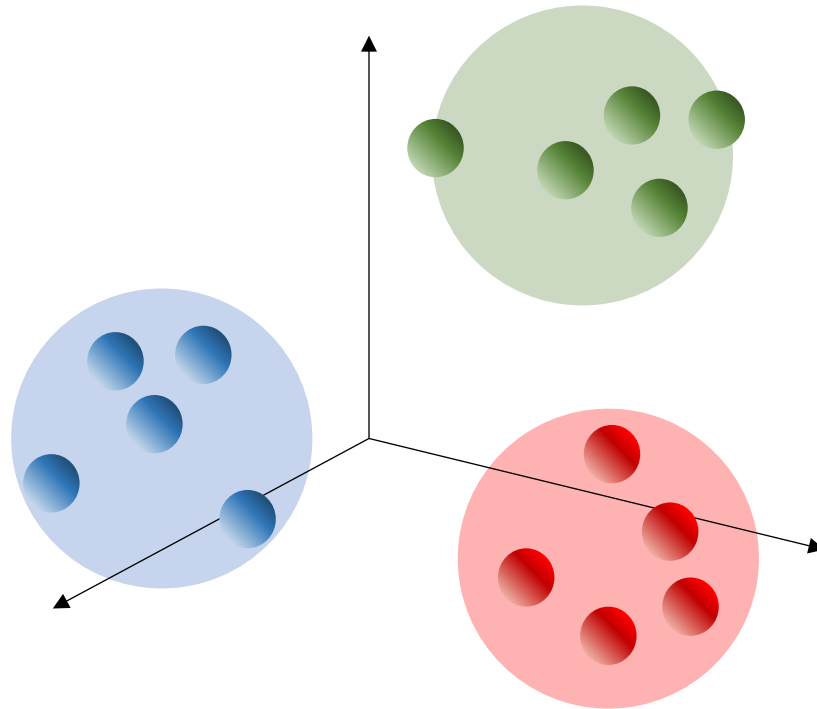
- Distance function
 - All data samples in a single cluster should share similar characteristics.
 - Any data samples in different clusters should be different.

Minimize the distance within each group.

=

Maximize the distance across groups.

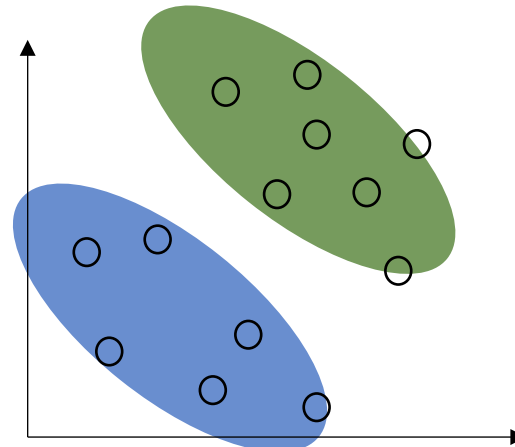
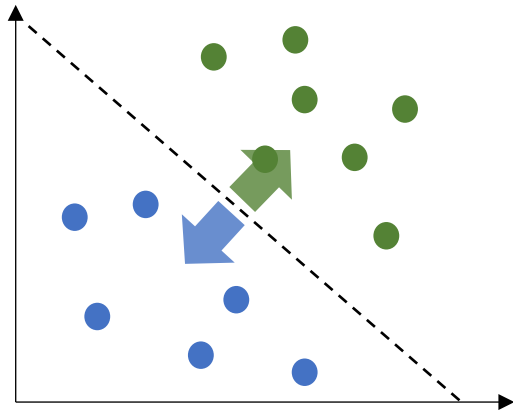
거리 함수 잘 정의시 사실상 최소화/최대화 문제로 치환 가능



문제와 데이터에 따라 적절히 정의해야함

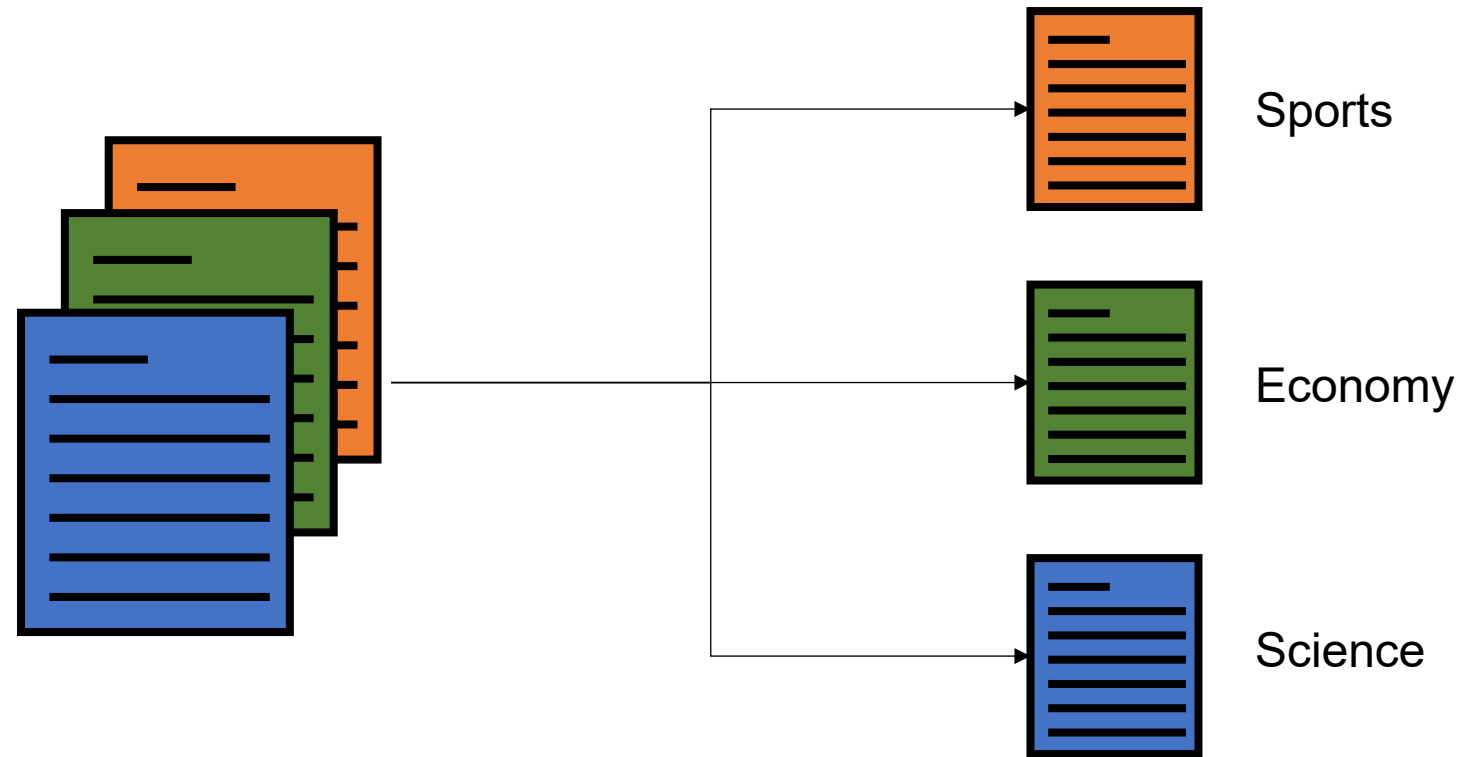
Classification vs. Clustering

- Classification: the model learns how to predict the 'right' class on each data sample. 올바른 정답에 해당하는 그룹을 할당
Supervised!
- Clustering: the models finds the best way of grouping the data.
Unsupervised!
가장 잘 구별되는 듯한 그룹을 할당 = 올바른 정답이 없음



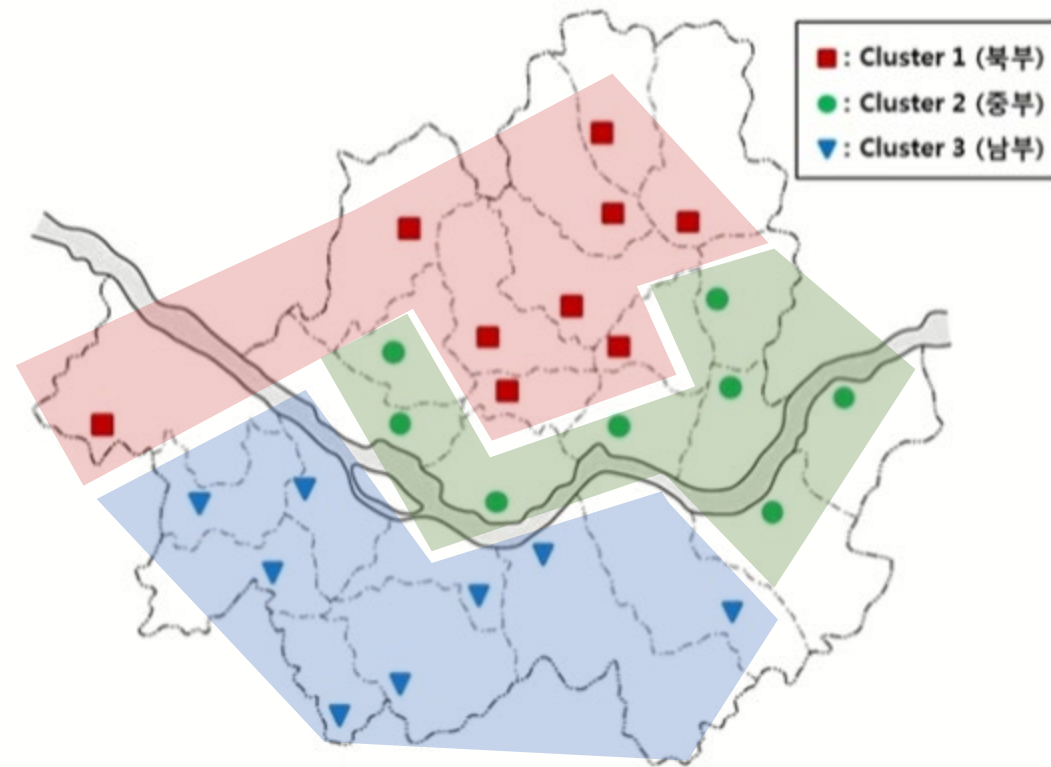
Clustering Applications (1/3)

- Article Clustering (Facebook postings, blog postings)



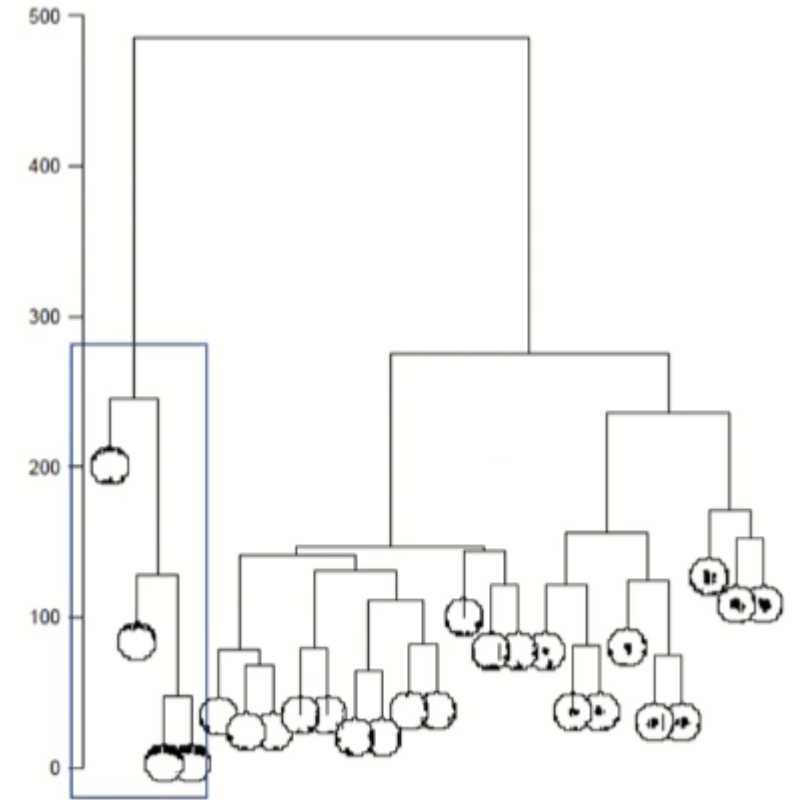
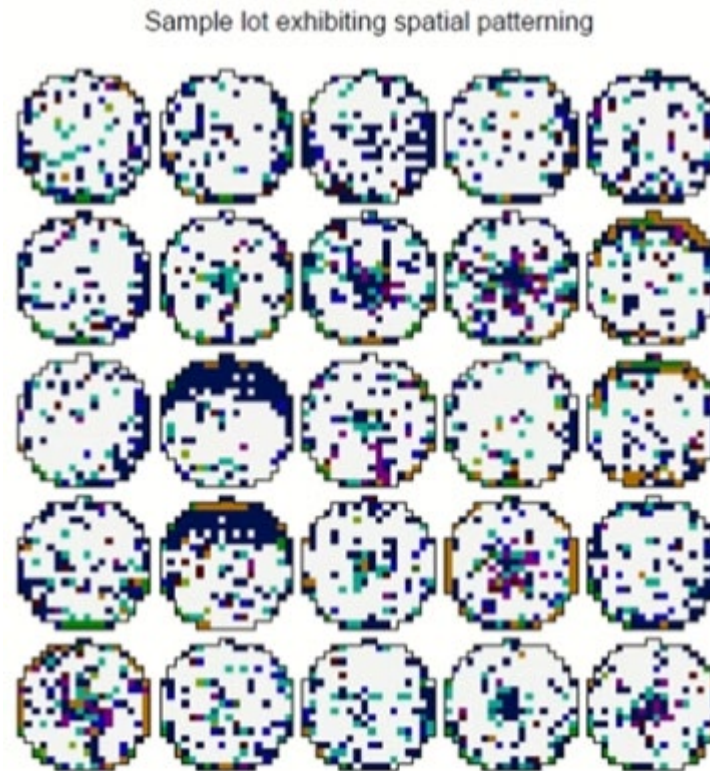
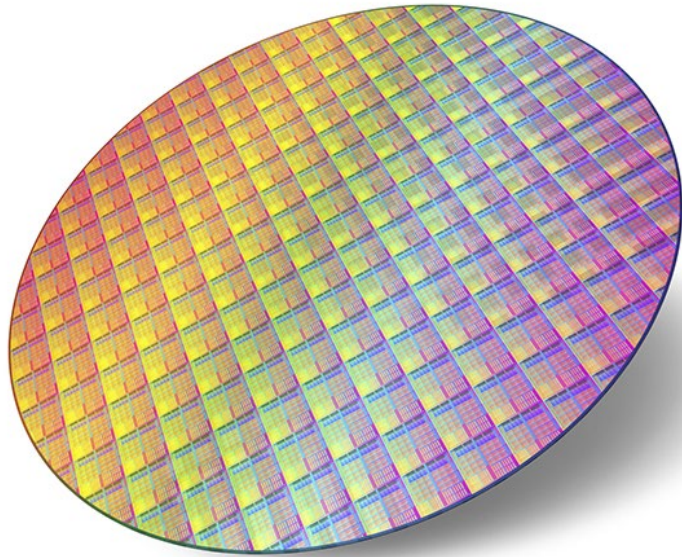
Clustering Applications (2/3)

- Pattern Recognition
 - Ozone Density Analysis (25 regions in Seoul)



Clustering Applications (3/3)

- Wafer Fail-bit Mapping



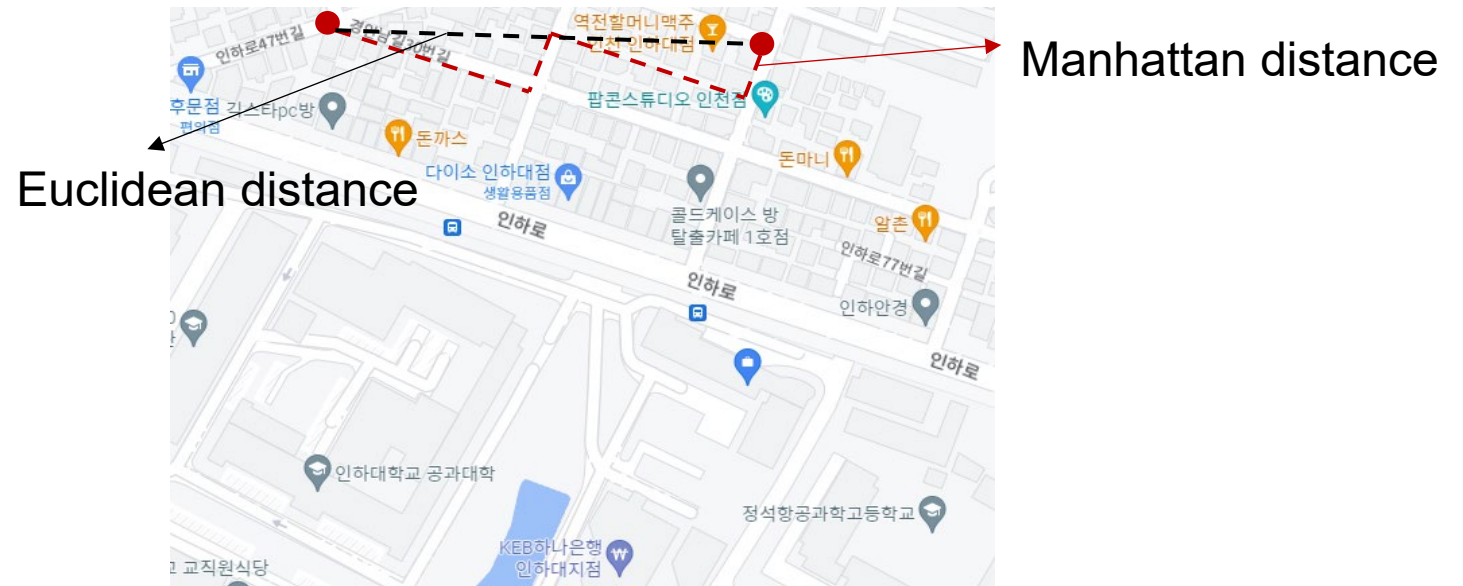
How to Cluster Data

- Questions:
 - How to measure the '***distance***' between any two data samples?
 - Which ***algorithm*** to find the clusters?
 - How ***many clusters*** to find?
 - How would you evaluate the ***quality*** of the output?

Distance Metrics

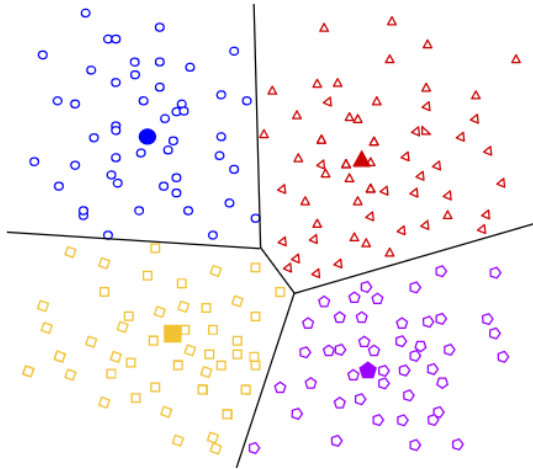
- We already took a look at several distance metrics:

- Euclidean
- Manhattan
- Cosine similarity
- Correlation
- Mahalanobis
- Etc.

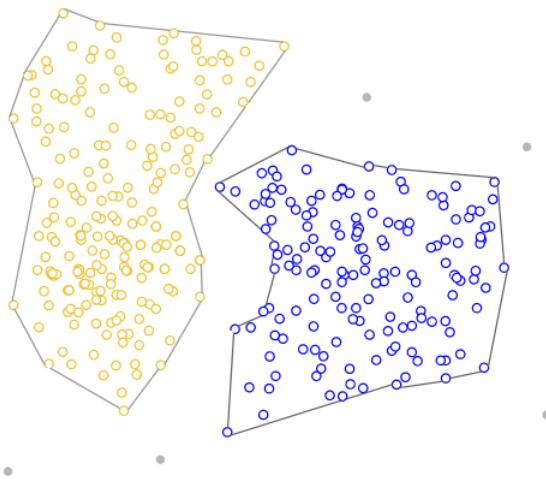


Clustering Algorithms

- Centroid-Based Clustering
- Density-Based Clustering
- Distribution-Based Clustering

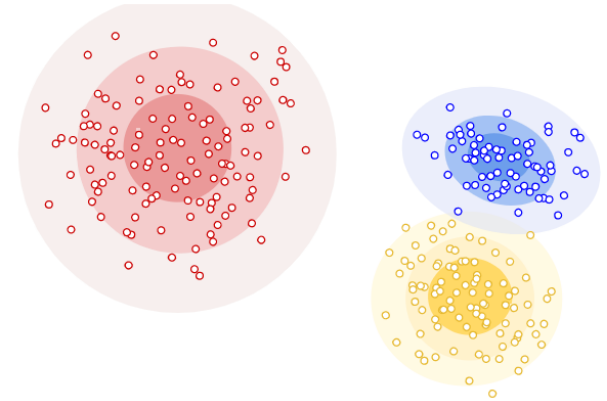


K-Means



DB-SCAN

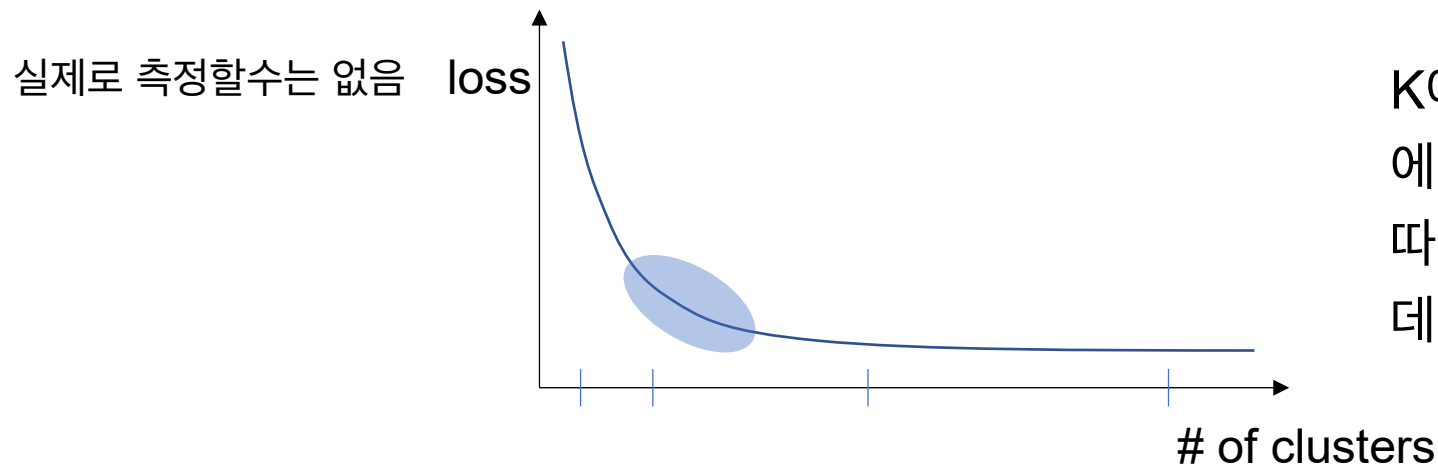
각 군집의 분포가 정규분포 = 비현
실적 => 활용가능한 상황이 적음



Gaussian Mixture

How Many Clusters?

- It depends on the data.
- Some algorithms automatically find the number of clusters.
 - E.g., DB-SCAN finds the number of clusters from the given data.
- Users can try several different settings and choose the best one.
 - The best setting usually appears at the elbow point.



K에 따른 성능 변동은 테스트 이후
에야 알 수 있으므로 보통은 성능에
따라결정하기 보다는 문제 정의와
데이터 특성에 따라 미리 결정

How to Evaluate the Performance

- There is no way to 'validate' the prediction (remember we do not have labels).
 - The 'accuracy' or 'error' could be used for classification tasks.
 - Clustering does not have such universal metrics.
- Users may empirically evaluate the quality of the predictions.
 - E.g., Sum of Squared Error (SSE)

$$SSE = \sum_{i=1}^K \sum_{x \in C_i} dist(x, c_i)^2$$

모든 군집에 대한 군집내 거리 재곱의 합을 더한 것

- Unsupervised Learning?
- Clustering
- ***Example: K-Means***
- Example: DBSCAN
- Example: Hierarchical Clustering

K-Means Clustering Algorithm

- A representative centroid-based clustering algorithm.
 - Each 'cluster' has a centroid.
 - Every data sample is assigned to a cluster whose centroid is the closest.
 - The number of clusters, K , is a hyper-parameter!

$$X = C_1 \cup C_2 \dots \cup C_K, \quad C_i \cap C_j = \emptyset, i \neq j$$

$$\arg \min_C \sum_{i=1}^K \sum_{x_j \in C_i} \text{dist}(x_j, c_i)$$

지도 학습이 필요하나 레이블이 없는 데이터에 대해 유사 레이블을 분류하고 싶을때 사용 가능

Algorithm

I와 J를 동시에 처리하는 과정이 없으므로 모든 데이터에 대한 병렬화 가능

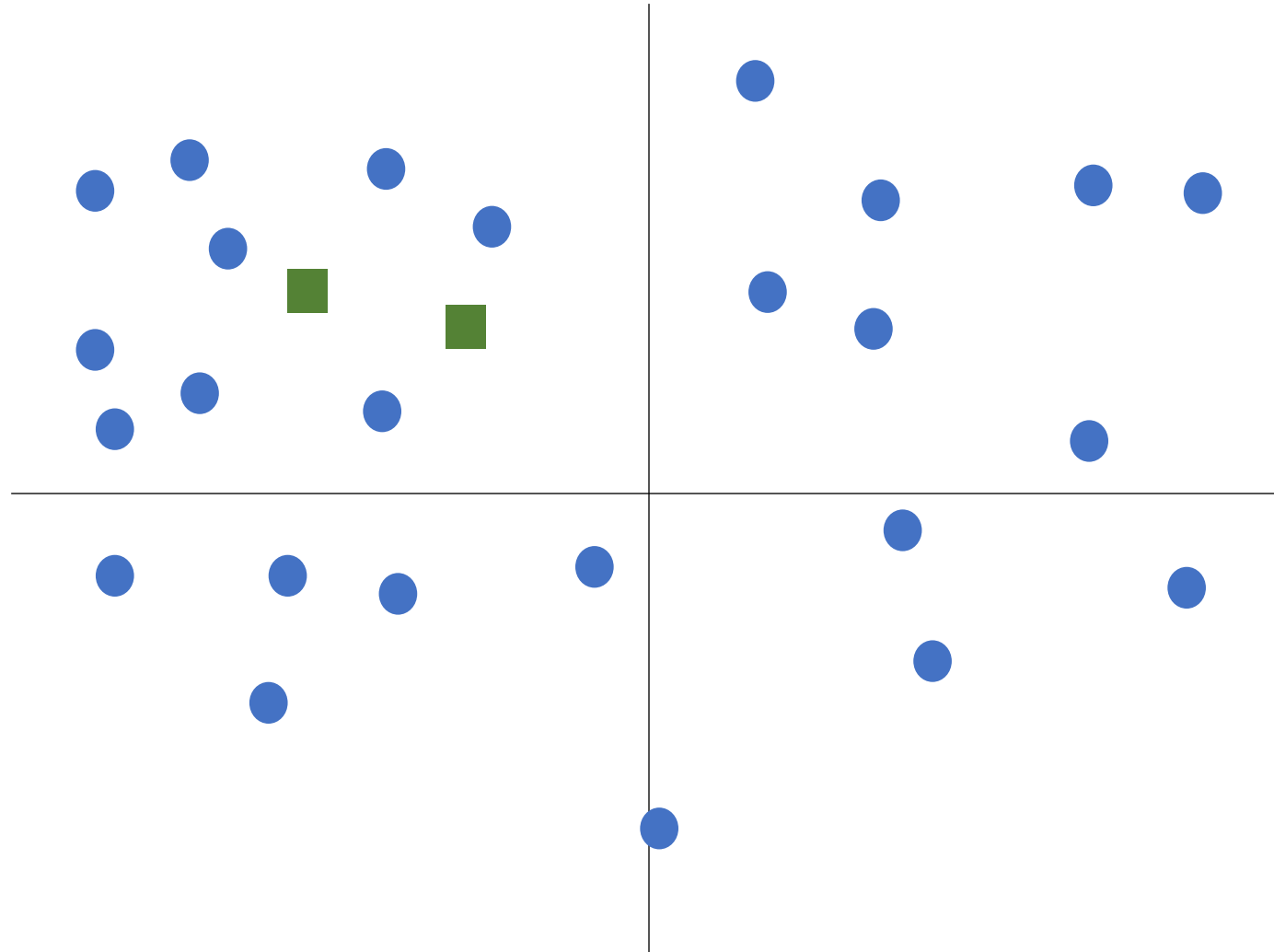
Algorithm K-Means Data Clustering Algorithm

1. **while** $\frac{\delta}{N} > threshold$ **do** 그룹 배정 결과가 잘 안바뀌면 군집화 종료
2. $\delta = 0$
3. **for** each data point $i \in N$ **do**
4. $d_{min} = FLOAT_MAX$
5. **for** each cluster $j \in K$ **do**
6. **if** $dist(x_i, c_j) < d_{min}$ **then**
7. $d_{min} = dist(x_i, c_j)$
8. $m = j$
9. **if** $membership[i] \neq m$ **then**
10. $\delta += 1$
11. $membership[i] = m$
12. **for** each cluster $j \in K$ **do**
13. $c_j \leftarrow$ mean of all the cluster members.

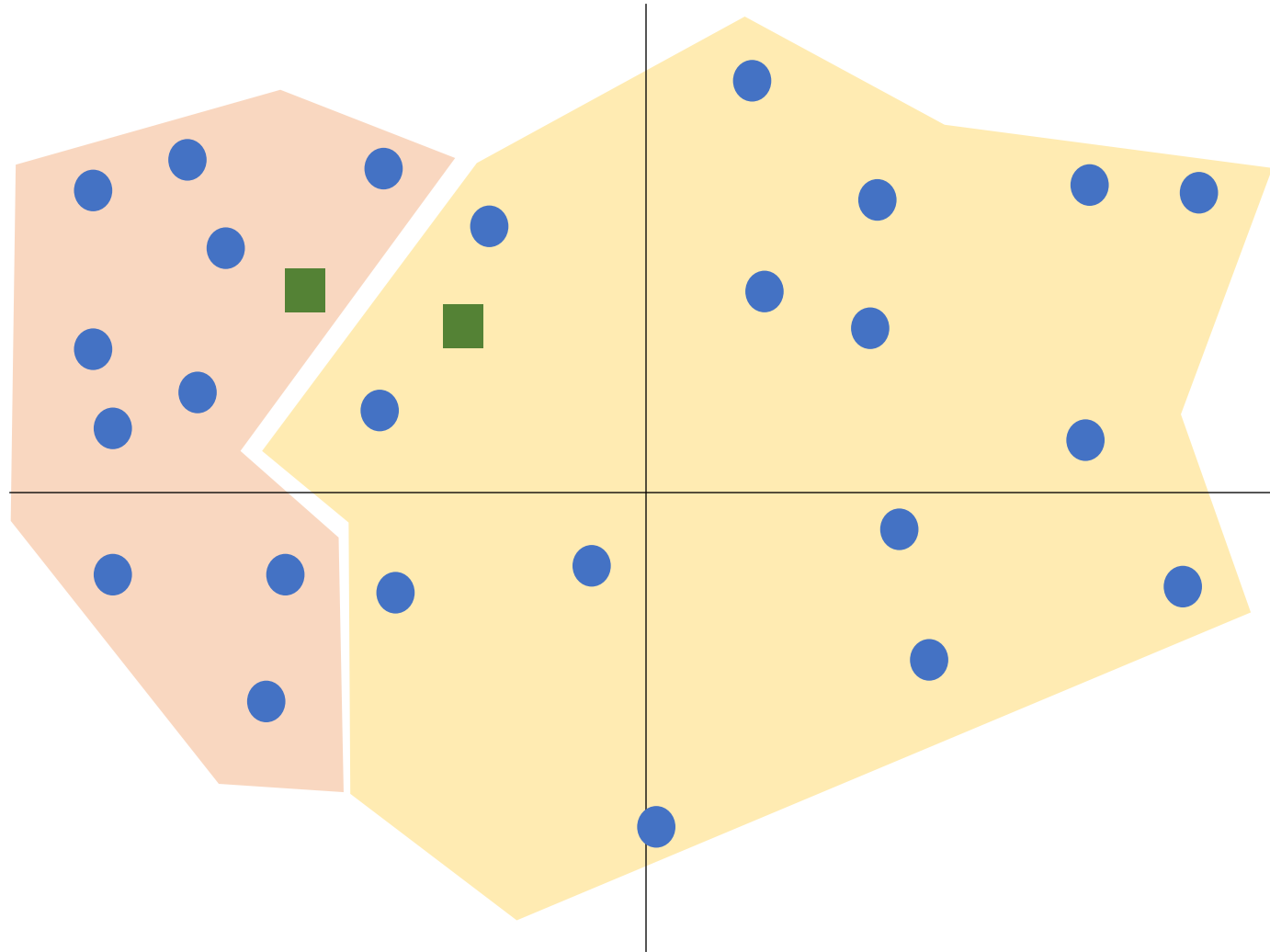
} Find the nearest centroid!

= 군집의 대표값 = 그 최초 기준을 보통 설문조사로 찾음

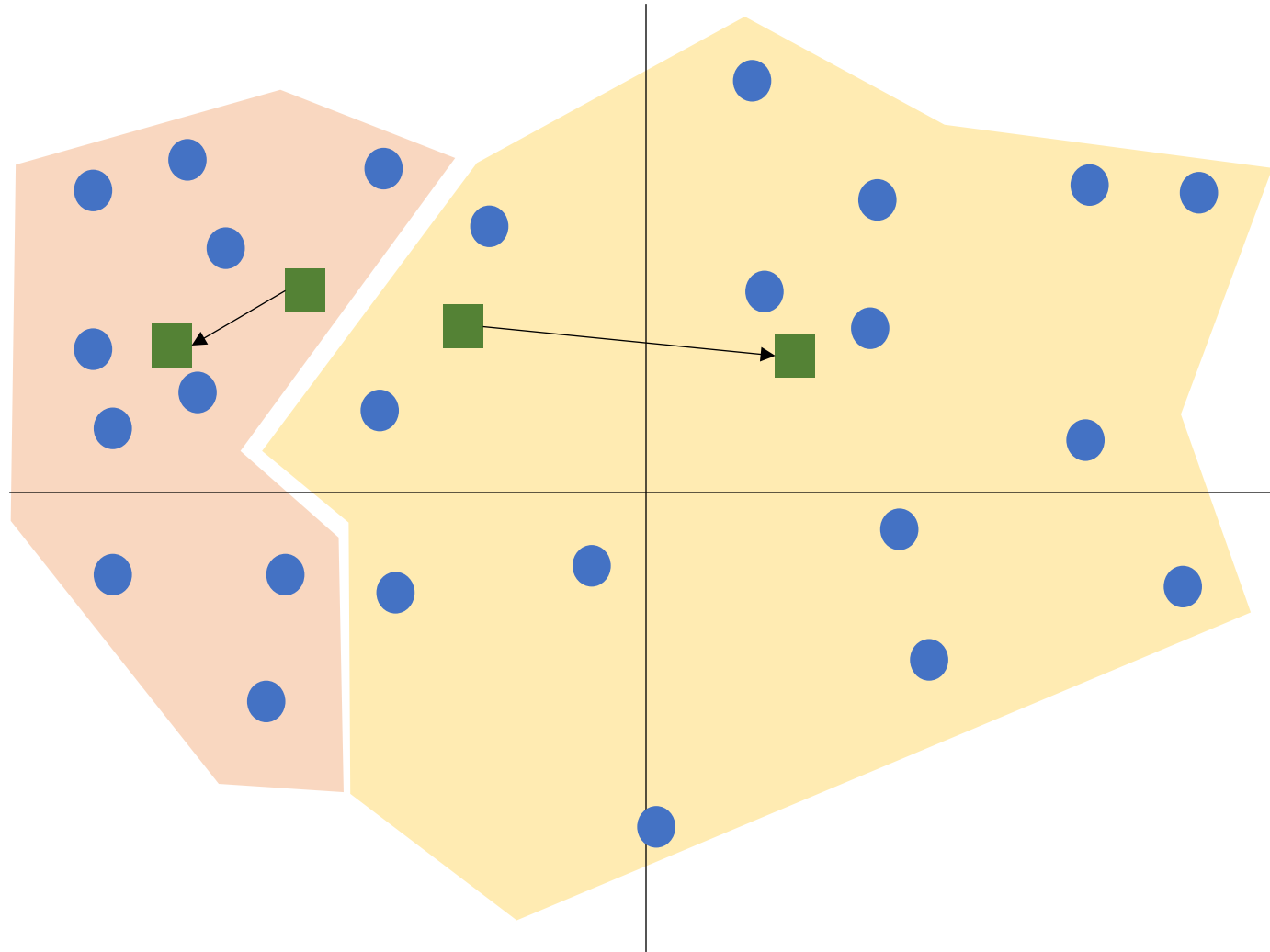
Example: K-Means Clustering (K=2)



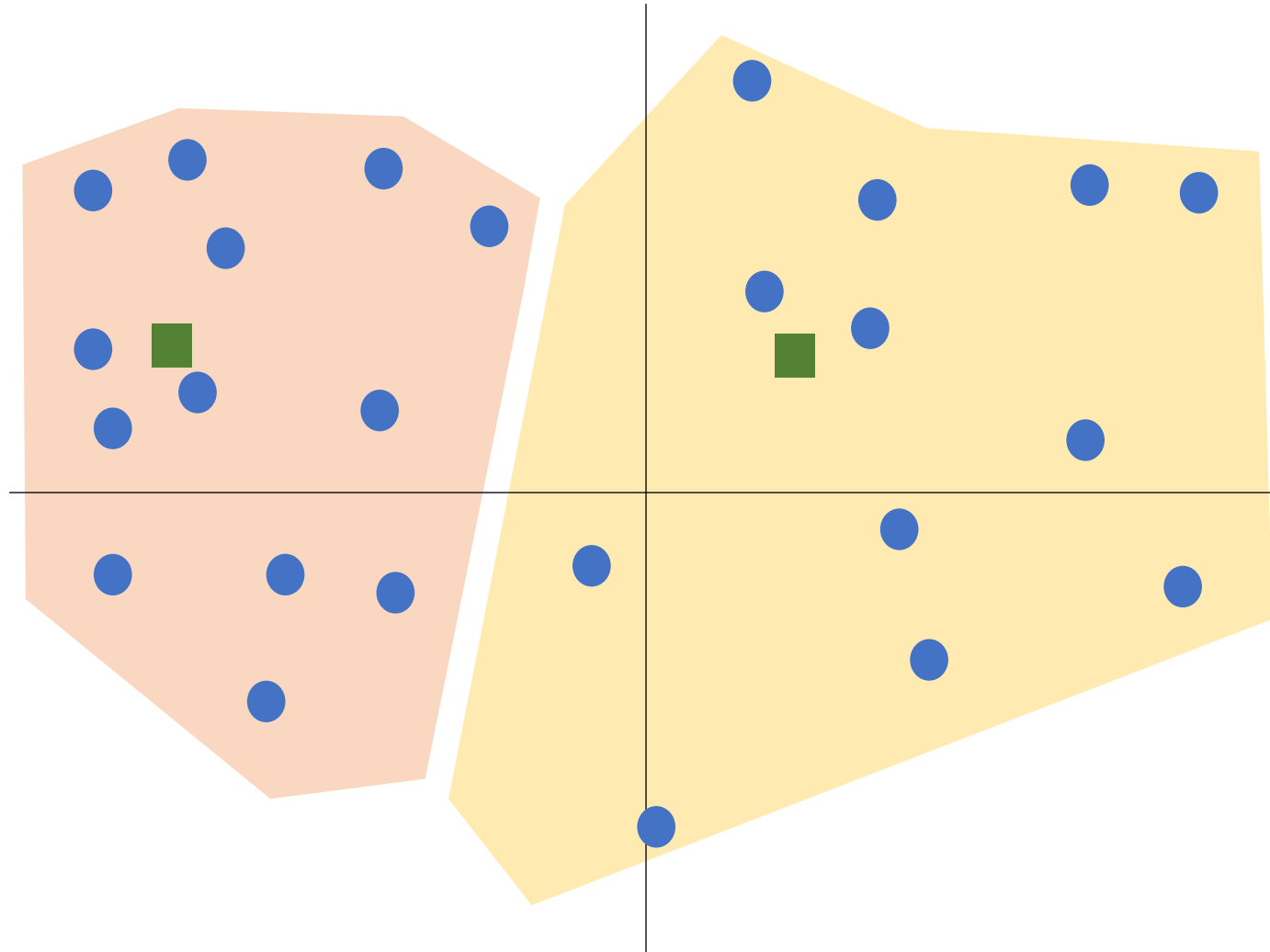
Example: K-Means Clustering (K=2)



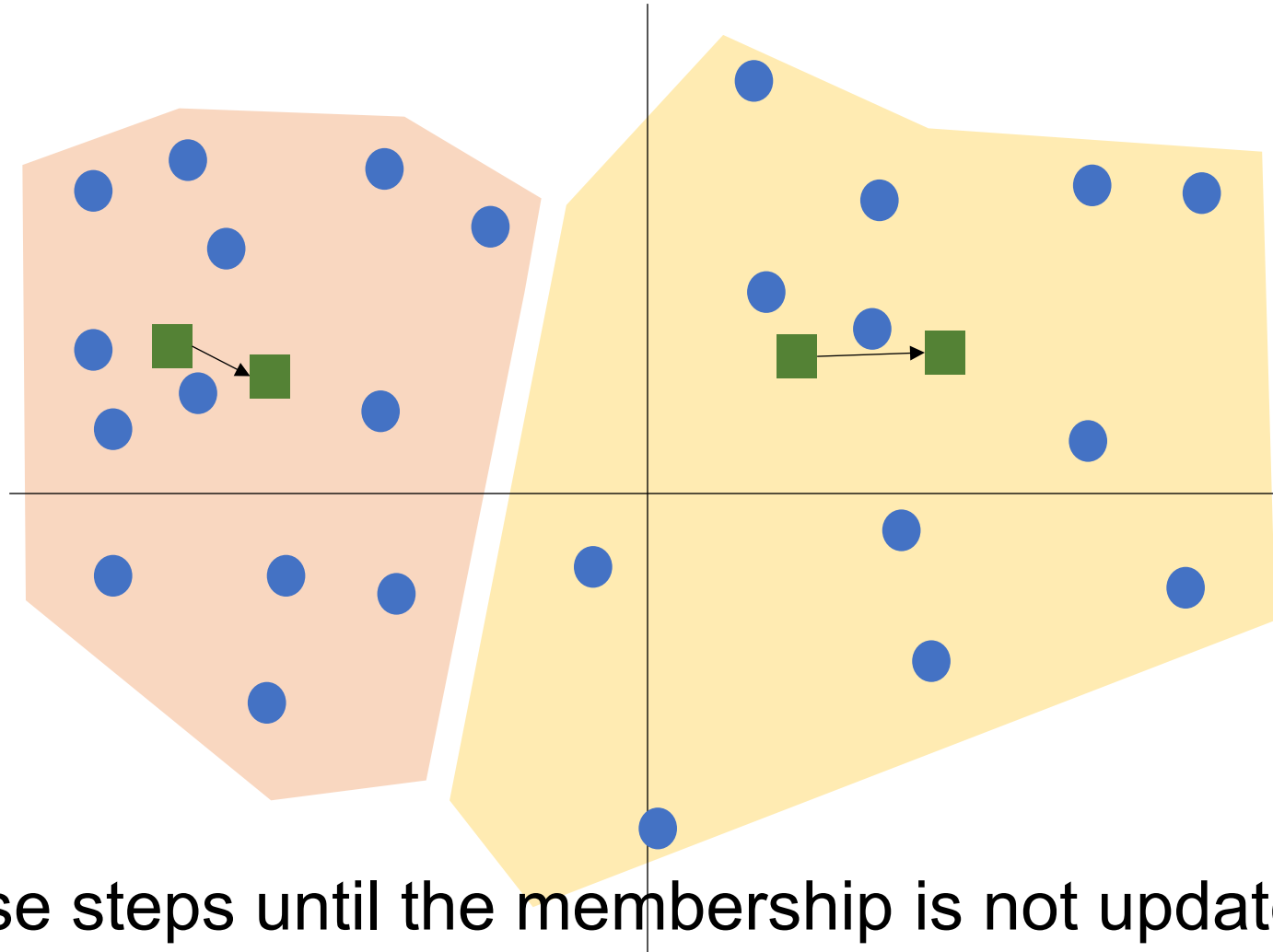
Example: K-Means Clustering (K=2)



Example: K-Means Clustering (K=2)

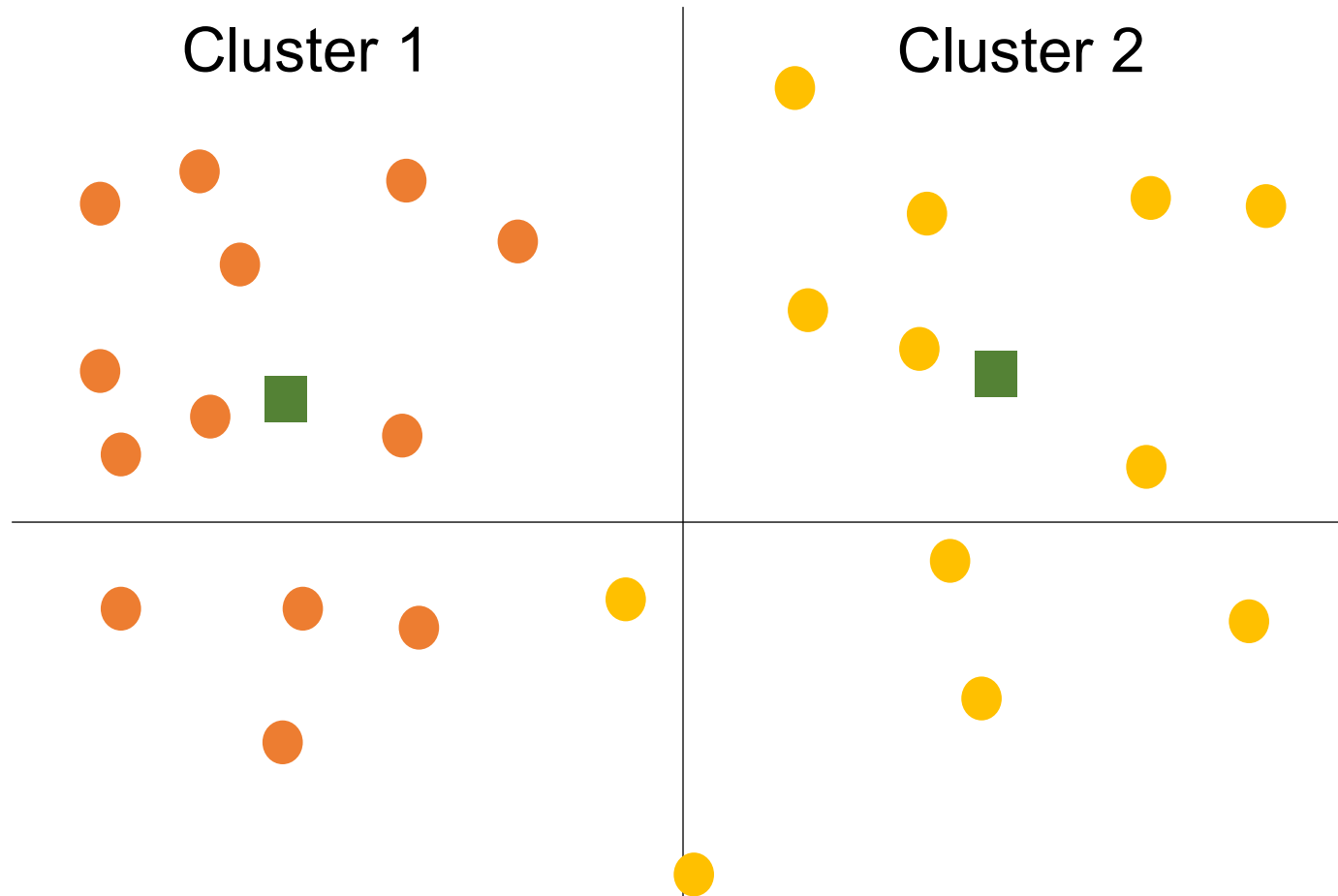


Example: K-Means Clustering (K=2)



Repeat these steps until the membership is not updated anymore!

Example: K-Means Clustering (K=2)

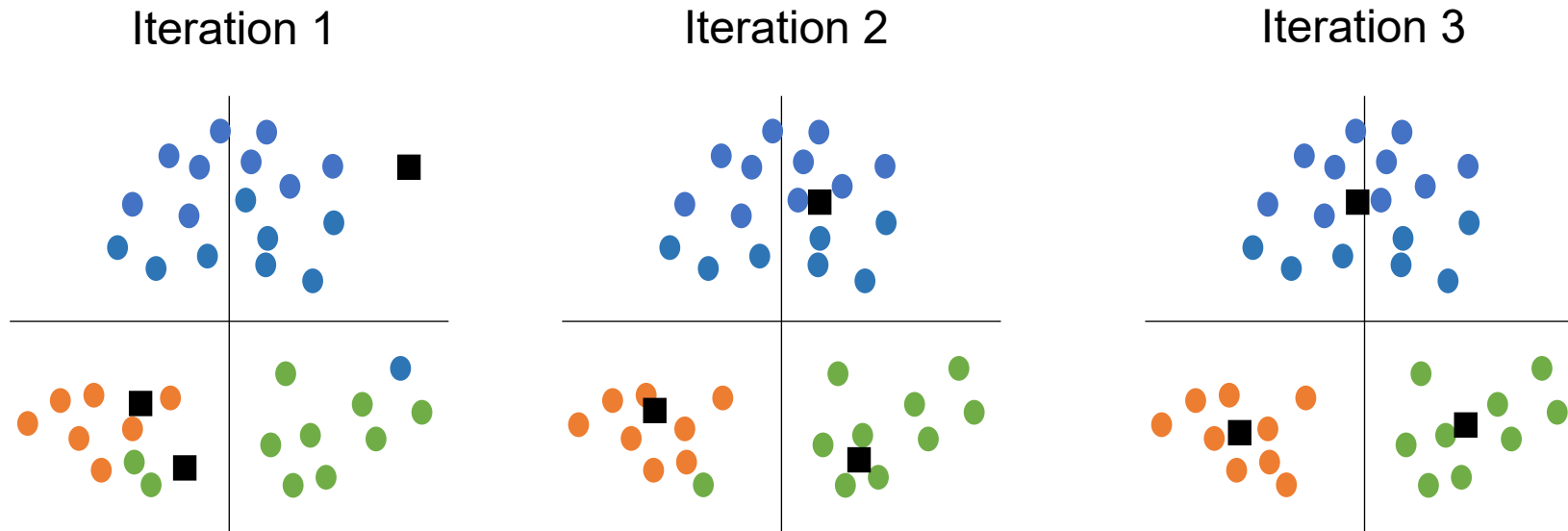


실제 데이터에 적합한
Centroid가 확정되므
로 각 데이터를 대표하
는 성질을 알 수 있음

Now, the algorithm has found two groups of 'close' data points.

Random Initial Centroid (1/3)

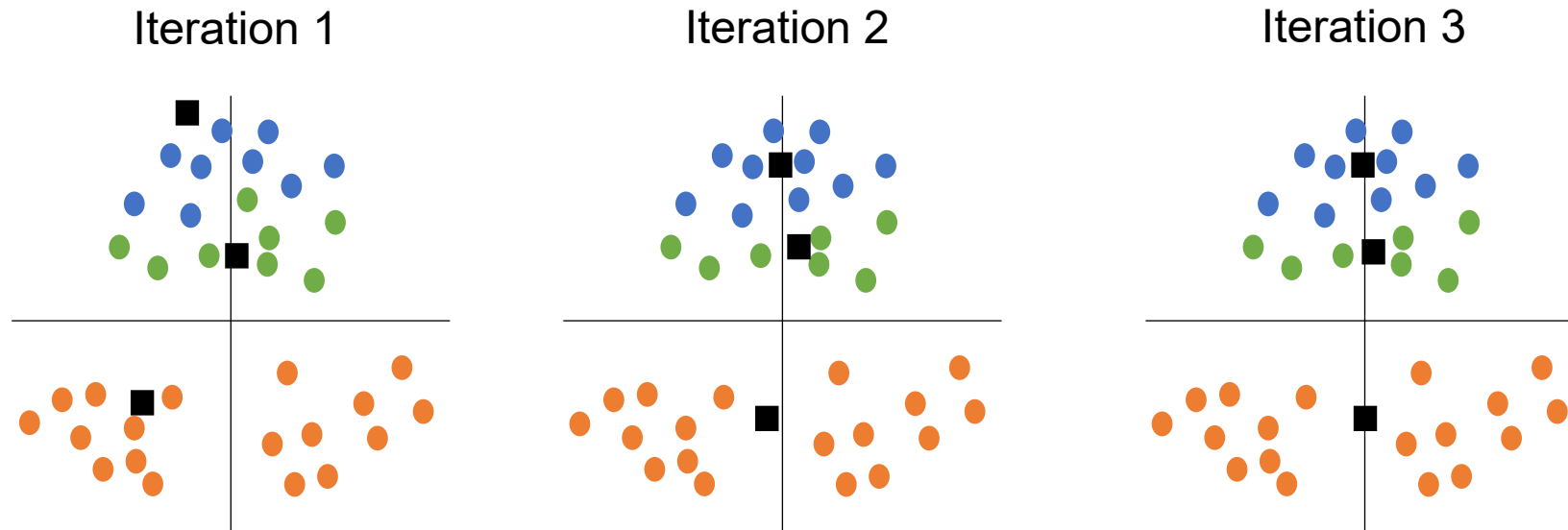
- Usually, the initial centroids are randomly selected.
- However, it affects the quality of training outputs!



A Successful Case

Random Initial Centroid (2/3)

- Human observes three noticeable groups, but the K-Means finds different results...



A Bad Case

Random Initial Centroid (3/3)

- A variety of on-going research works that try to tackle the issue of erroneous random points.
 - The easiest way to avoid such an issue is to *run K-Means multiple times* and choose the most frequently appearing clusters.
 - One can also *make use of the priori information* (the data locations in our case), and choose better random initial points.
- However, when the dataset size is large, this issue usually does not make a big impact on the final output.

임의 시작점 자체를 조절하는 방법은 순간 편향이 발생하므로 여러 번 K-Means 수행 후 이상치를 제외한 Normal한 케이스를 결과값으로 채택하여 해결해야함

From Optimization's Perspective...

- K-Means clustering is a minimization process.
 - We want to minimize the average distance between each point and its closest centroid.
 - The loss function naturally becomes MSE.
 - Consequently, the gradient of the loss function is the average distance between the centroid and the cluster members.

From Optimization's Perspective...

- Loss function: $L = \sum_{i=1}^N \sum_{j=1}^K a_{i,j} \|x_i - c_j\|^2$.

- $$\frac{\partial L}{\partial c_j} = \frac{\partial \sum_{i=1}^N a_{i,j} \|x_i - c_j\|^2}{\partial c_j}$$
$$= \frac{\partial \sum_{i=1}^N a_{i,j} (x_i^2 - 2x_i c_j + c_j^2)}{\partial c_j}$$

$a_{i,j}$ is 1 if x_i belongs to cluster j , otherwise 0.

$$= \sum_{i=1}^N a_{i,j} (-2x_i + 2c_j) = 0$$



$$c_j = \frac{\sum_{i=1}^N a_{i,j} x_i}{\sum_{i=1}^N a_{i,j}}$$

Average point of all the cluster members!

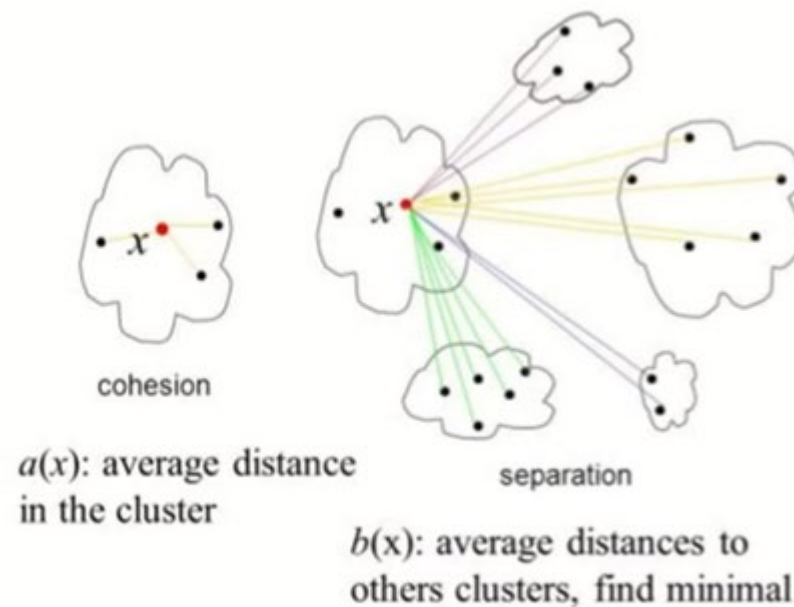
Silhouette Score

- One popular evaluation method for clustering methods.
 - $a(x_i)$: the average distance from x_i to all the same cluster members.
 - $b(x_i)$: the minimum average distance from x_i to other clusters.

$$s(x_i) = \frac{b(x_i) - a(x_i)}{\max\{a(x_i), b(x_i)\}}, -1 \leq s(x_i) \leq 1$$

$$\bar{s} = \frac{1}{N} \sum_{i=1}^N s(x_i)$$

- In general, $\bar{s} \geq 0.5$ implies the clustering is successful.
- $\bar{s} \approx -1$ means it was not clustered at all.



K-Means Pros & Cons

- ↑ • Easy to implement
- ↑ • Intuitive (good interpretability)
- ↑ • Good scalability 비지도 학습의 경우 정답이 없으므로 입력값의 크기가 보통 큼, 따라서 병렬화 성능 중요
- ↑ • Guarantees convergence.
- ↓ • Hard to find a good K
- ↓ • Initial value may affects the output quality.
- ↓ • Weak to outliers. : 모든 데이터가 예외없이동등하게 평균 거리에 영향을 미치기 때문
- ↓ • Does not work for various dimensional data.

- Unsupervised Learning?
- Clustering
- Example: K-Means
- ***Example: DBSCAN***
- Example: Hierarchical Clustering

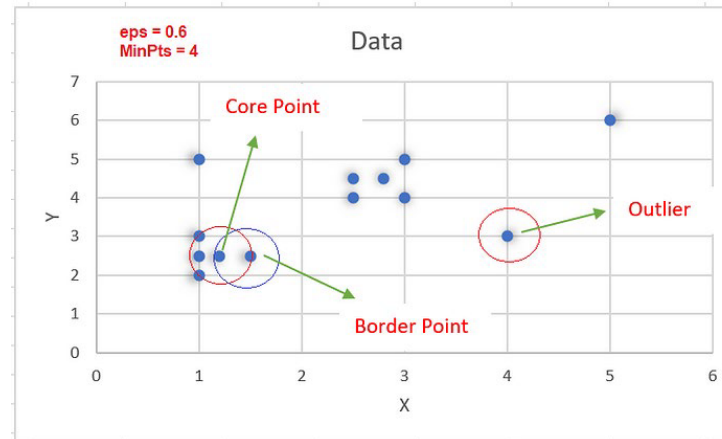
Hyper-Parameter tuning is challenging..

- What if we do not know which k is the best?



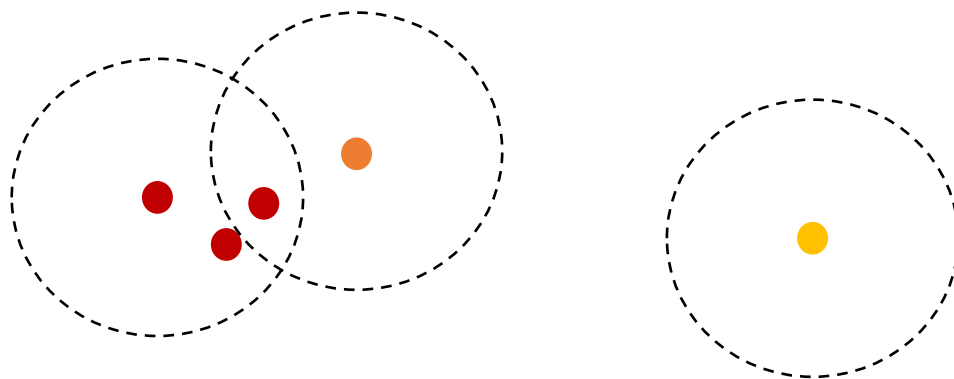
Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

- A representative density-based clustering method.
- Two optional hyper-parameters:
 - ϵ : The maximum distance between 2 points for them to be considered as neighbors. 일정 거리이내에 있는 데이터가 밀도를 형성하며, 같은 클러스터라고 인정
 - *min_samples*: The minimum number of neighbors a point needs to have to be considered as a 'core' point. 각 클러스터에 최소 샘플 수 이상의 데이터가 있어야함



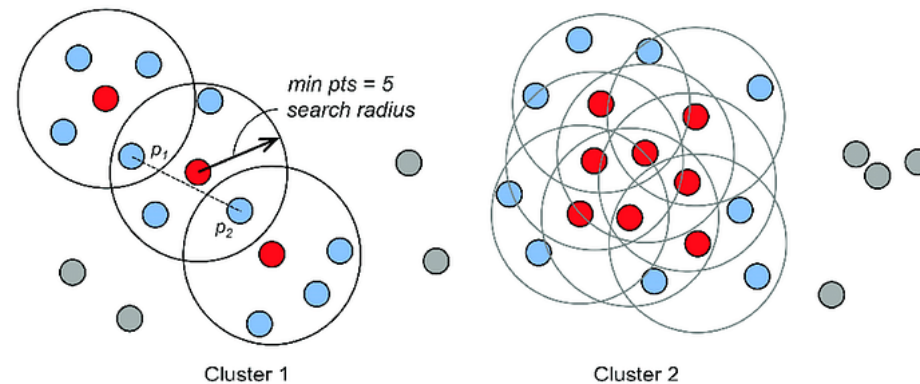
Types of Points

- **Core points** 거리 조건을 만족하여 밀도를 형성하고 최소 샘플 수 이상
 - All data points whose neighbors are more than *min_samples*.
- **Border points** 거리 조건을 만족하여 밀도를 형성하나 최소 샘플 수 미만
 - All data points whose neighbors are less than *min_samples* but more than one.
- **Noise points** 거리 조건을 만족하지 못하여 밀도를 형성하지 못 하고 최소 샘플 수 미만
 - All data points who do not have neighbors within $< \epsilon$.



Algorithm

1. Determine 'core' points.
2. Traverse over all individual core points.
 1. If the core point has not been assigned with any cluster, create a new cluster.
 2. Spread the cluster to the close points (distance $< \epsilon$).
3. Repeat the step 2 until there are no core points without a cluster.

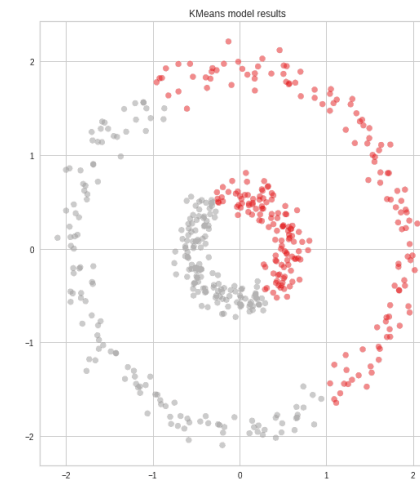
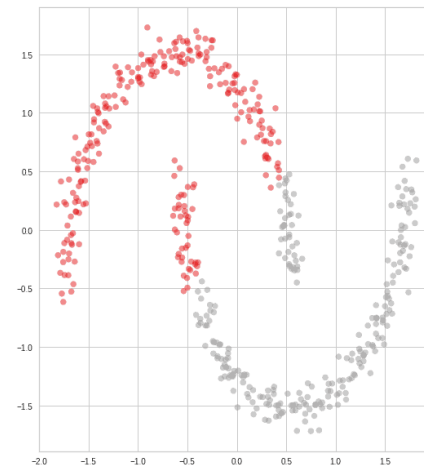
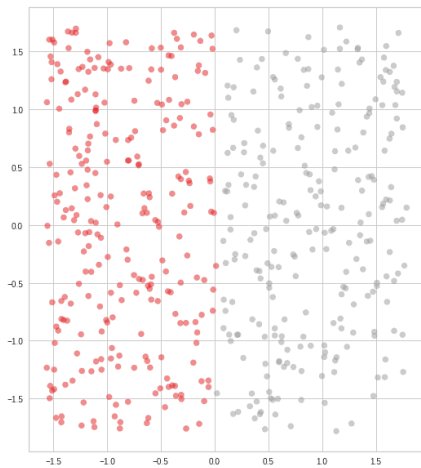


DBSCAN

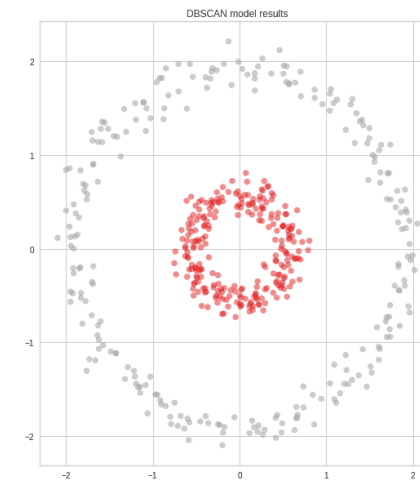
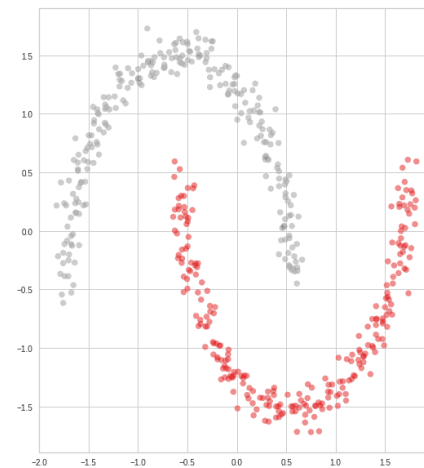
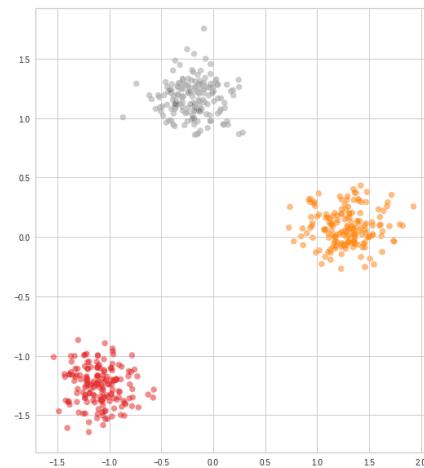
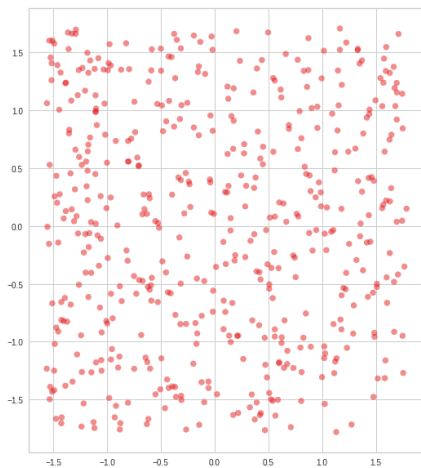
Algorithm 1 The DBSCAN algorithm. Input: A set of points X , distance threshold eps , and the minimum number of points required to form a cluster, $minpts$. Output: A set of clusters.

```
1: procedure DBSCAN( $X, eps, minpts$ )
2:   for each unvisited point  $x \in X$  do
3:     mark  $x$  as visited
4:      $N \leftarrow \text{GETNEIGHBORS}(x, eps)$ 
5:     if  $|N| < minpts$  then
6:       mark  $x$  as noise
7:     else
8:        $C \leftarrow \{x\}$ 
9:       for each point  $x' \in N$  do
10:         $N \leftarrow N \setminus x'$ 
11:        if  $x'$  is not visited then
12:          mark  $x'$  as visited
13:           $N' \leftarrow \text{GETNEIGHBORS}(x', eps)$ 
14:          if  $|N'| \geq minpts$  then
15:             $N \leftarrow N \cup N'$ 
16:          if  $x'$  is not yet member of any cluster then
17:             $C \leftarrow C \cup \{x'\}$ 
```

- Published by Ester et al. in 1996.
- The complexity of this algorithm is $O(n^2)$, where n is the number of data points.
- With spatial indexing (KD-tree or R-tree), the complexity becomes $O(n \log n)$.



K-Means



DBSCAN

DBSCAN Pros & Cons

- ↑ • Generally works well.
- ↑ • Does not require a-priori knowledge about # of clusters.
- ↑ • Able to identify outliers.

밀도가 서로 다른 두 군집을 잘 구분하지 못 함, 엡실론은 하나로 상수이므로

- ↓ • If a cluster consists of a varying density, it may fail.
- ↓ • Fails on 'neck-type' clusters.
- ↓ • Does not scale well if data is too high-dimensional.

고차원 데이터에 취약, 따라서 병렬화 활용하는 경우 많음

Neck-type cluster

